

PLACIFY
AUTONOMOUS AI PLACEMENT ORCHESTRATION

AD23731 Foundations of Agentic AI - Mini-Project

Submitted by:

SHRI DHARSHINI M (231501153)

SHARAN M (231501151)

SOORYANATH SD (231501158)

**Department of Artificial Intelligence
And
Machine Learning**

(An Autonomous Institution, Affiliated to Anna University, Chennai)

SEPTEMBER 2026

BONAFIDE CERTIFICATE

Certified that this mini-project report “PLACIFY – AUTONOMOUS AI PLACEMENT ORCHESTRATION” is the bonafide work of “Shri Dharshini M (231501153), Sharan M (231501151), Sooryanath SD (231501158)” VII Semester, B.Tech. Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course AD23731 – Foundations of Agentic AI during the Academic Year 2026 – 2027.

Faculty in-charge

Submitted to mini-project viva voce examination held on:
.....

INTERNAL EXAMINER	EXTERNAL EXAMINER

INDEX

No.	CHAPTERS	Page No.
1	Abstract	1
2	Introduction	2
3	Problem Statement and Business Case	4
4	User and Stakeholder Analysis	5
5	Proposed Agentic AI Solution	7
6	Dataset / Knowledge Sources	9
7	Tools and Technologies Used	10
8	Agent Architecture	12
9	Agent Workflow and Orchestration	14
10	Agentic RAG Implementation	16
11	System Implementation	18
12	Experimental Evaluation and Results	22
13	Comparison, Innovation and Scalability	24
14	Conclusion and Future Work	27

ABSTRACT

Placify is an Agentic AI based placement orchestration system designed to reduce the manual coordination involved in campus recruitment. The system treats a placement drive as a continuously changing workflow rather than a collection of independent forms, emails and spreadsheets. When a company provides a job description and eligibility criteria, Placify converts the requirements into structured conditions, identifies potentially eligible students, verifies relevant academic information through an authorized result source, initiates the application process and tracks responses. Specialized agents then handle communication, assessment-result processing, shortlisting and interview scheduling. A scheduling agent checks examinations, other placement drives, available interview slots and relevant availability before proposing a schedule. A conflict-checking and reflection stage validates the proposed plan. If HR changes a requirement or interview timing, the workflow can identify the affected state, recalculate the schedule and trigger re-planning instead of restarting the entire drive.

The system is coordinated by a stateful LangGraph orchestrator. The Company Agent, Eligibility Agent, Verification Agent, Application Agent, Communication Agent, Assessment Agent, Shortlisting Agent, Scheduling Agent, Conflict Checker and Critic/Reflection Agent have separate responsibilities but operate on shared placement state. Agentic RAG provides access to placement policies, eligibility rules, job descriptions and institutional guidance when an agent needs additional context. Short-term memory maintains the current drive state, while long-term memory stores relevant historical drive information and decisions. Tool integration is provided through backend services and authorized external interfaces such as form, email, notification, academic result and calendar services. PostgreSQL or an equivalent relational store can maintain structured placement records, while a vector database supports semantic retrieval.

The project is intended as an academic prototype demonstrating how Agentic AI can connect reasoning, retrieval, tool use, memory, reflection and human approval into one practical college workflow. The system does not assume unrestricted access to a real college result portal; production deployment would require authorized APIs, institutional permissions, security controls

and validation. The central contribution is the ability to move from company requirements to coordinated placement actions while retaining traceability and human oversight for critical decisions.

1. Introduction

1.1 Background and Motivation

Campus placement is a multi-stage process involving students, placement officers, faculty coordinators and company recruiters. A typical drive begins when a company shares a job description and a set of eligibility conditions. The placement team must interpret these conditions, identify suitable students, verify academic information, communicate the opportunity, collect applications, process assessments, prepare a shortlist and finally coordinate interviews. Each stage depends on the output of the previous stage, yet many colleges still manage these activities through separate forms, spreadsheets, email threads, messaging groups and manually maintained schedules.

The difficulty increases when a company specifies several conditions at the same time. Eligibility may depend on department, CGPA, school marks, arrears, graduation status, skills or other institution-approved criteria. A student may appear eligible from one record but fail a condition in another source. Similarly, an applicant may submit a form but later become unavailable because of an examination or another placement activity. Manual coordination can therefore create repeated checking and communication work for the placement team.

Placify is motivated by the idea that placement coordination can be represented as a stateful workflow that an intelligent system can observe, reason over and update. The objective is not to replace the placement officer, but to automate repetitive coordination and provide an adaptive decision-support layer. The system can determine what action is needed next, select the relevant specialist agent, use an appropriate tool or knowledge source, validate the result and update the current placement state.

1.2 Placement Process as a Workflow

The proposed system models a placement drive as a sequence of connected states: requirement received, eligibility evaluated, verification completed, applications open, applications tracked, assessment processed, shortlist prepared, interview planned, conflicts checked and final schedule approved. This representation is important because a change at one stage can affect later stages. For example, a change in the company's eligibility condition may

invalidate a previous shortlist, while a change in interview timing may affect several students and rooms.

1.3 Need for Agentic AI

A conventional automation system can execute predefined rules, but campus placement contains many situations where the next action depends on the current state. An agentic system can combine structured rules with reasoning, retrieval and tool use. For example, when a company JD is received, the Company Agent can identify eligibility constraints. The Eligibility Agent can then query the student data source, while the Verification Agent checks the evidence required for a final decision. After applications are collected, the Communication Agent can identify pending students and issue targeted reminders.

The agentic approach is especially useful for schedule management. A scheduling agent must consider multiple calendars and constraints rather than simply assigning the next free time. A reflection agent can inspect the proposed schedule and return the workflow for re-planning when a conflict is detected. This creates a closed decision loop instead of a one-way automation script.

1.4 Project Objectives

- Develop a stateful multi-agent architecture for campus placement orchestration.
- Convert company job descriptions and criteria into structured placement requirements.
- Identify and verify eligible students using authorized academic information.
- Automate application initiation, status tracking and targeted reminders.
- Process authorized assessment results and generate an auditable shortlist.
- Create interview schedules while considering examinations and other placement events.
- Detect conflicts and support adaptive re-planning when conditions change.
- Use Agentic RAG and memory to provide context-aware placement decisions.
- Preserve human approval for sensitive or high-impact placement actions.

1.5 Scope of the Project

The academic prototype covers the digital coordination layer of a placement drive. It focuses on data intake, eligibility, verification, communication, assessment handling, shortlisting and scheduling. It does not claim unrestricted access to institutional systems. Any connection to a real college result portal, student information system or company assessment platform would require authorization and a suitable API. The prototype can therefore use controlled datasets and simulated service responses while preserving the same agentic workflow that would be used in an institutional deployment.

2. Problem Statement and Business Case

2.1 Problem Definition

Campus placement coordination often involves repetitive manual work across multiple people and systems. When a company visits a college, the placement team must interpret the JD, check eligibility, prepare or configure an application form, notify eligible students, monitor responses, follow up with non-respondents, collect assessment results, prepare a shortlist and coordinate interview schedules. The same information may be entered or verified multiple times.

The core problem addressed by Placify is the lack of an adaptive coordination layer between company requirements, student information, placement communication and scheduling. A normal placement portal may store applications, but storage alone does not decide which action should happen next, which data source should be consulted or how to recover when an external condition changes.

2.2 Major Operational Problems

- Manual interpretation of company-specific eligibility criteria.
- Repeated cross-checking of student academic information.
- Separate preparation of forms, emails and reminder messages.
- Difficulty tracking eligible students who have not applied.
- Manual consolidation of assessment results and application status.
- Repeated effort in preparing shortlists under multiple conditions.
- Interview clashes with examinations, classes or other placement drives.
- Last-minute HR changes requiring manual rescheduling and communication.
- Limited visibility into the current state of a placement drive.
- Loss of traceability when decisions are spread across messages and spreadsheets.

2.3 Business / Institutional Context

For a placement cell, the value of automation is primarily coordination efficiency and reliability. A company drive may involve hundreds of students and several stakeholders. A system that automatically routes routine work can allow placement officers to focus on exceptions, approvals and communication with recruiters. For students, the same system can reduce missed opportunities caused by delayed notifications or unclear application status. For recruiters, a structured final list and organized schedule can reduce the effort required to coordinate with the institution.

3. User and Stakeholder Analysis

3.1 Target Users

Placify is intended for four main user groups. Students interact with the system to receive relevant opportunities, submit applications and view updates. Placement officers or TPOs initiate drives, review eligibility rules, monitor progress and approve important actions. HR or recruiters receive verified candidate lists, assessment outcomes and interview schedules. Faculty or placement coordinators provide supporting schedules and help manage institutional constraints.

3.2 Stakeholder Identification

Stakeholder	Role / Interest	Key Interaction with System
Student	Applies for relevant opportunities	Receives invitations, reminders, status and schedules
Placement Officer / TPO	Coordinates placement drives	Creates drives, reviews exceptions, approves lists and schedules
HR / Recruiter	Provides company requirements and recruitment outcomes	Views verified candidates, assessment status and interview schedule
Faculty / Coordinator	Supports academic and schedule coordination	Provides exam/event constraints and monitors conflicts
Placement Administrator	Maintains system configuration	Manages users, integrations, policies and audit records
Project Reviewer / Faculty Panel	Evaluates technical design	Reviews architecture, agent coordination, workflow and results

3.3 Stakeholder Needs

Students need timely and accurate communication. TPOs need visibility, control and auditability. Recruiters need a consistent candidate pipeline and reliable schedules. Faculty coordinators need conflict information.

Administrators need secure access control and traceable system actions. Placify addresses these needs through role-aware access, shared state, automated communication and human approval.

4. Proposed Agentic AI Solution

4.1 Overall Solution

Placify is organized as a stateful multi-agent solution coordinated by a central orchestrator. The orchestrator maintains the current placement-drive state and routes tasks to specialist agents. The Company Agent interprets the job description, the Eligibility Agent identifies matching students, and the Verification Agent checks academic evidence. The Application and Communication Agents initiate and manage the student-facing application process.

After applications close or an assessment is completed, the Assessment Agent processes authorized results. The Shortlisting Agent applies approved criteria and prepares the candidate set. The Scheduling Agent then constructs interview slots while considering examinations, other drives and availability. A Conflict Checker validates the schedule. The Critic/Reflection Agent reviews the output and can return the workflow to a previous stage for re-planning when necessary.

This design makes Placify more than a form-generation system. The agents share state, use tools and retrieve knowledge when needed. Human approval remains available for critical decisions such as final shortlist confirmation or schedule release.

4.2 Key Features

Company	Requirement	Understanding
The Company Agent extracts role, department, academic, skill and assessment conditions from a JD or structured recruiter input. The result is represented as explicit fields so that later agents can use the same criteria consistently.		

Eligibility	and	Verification
The Eligibility Agent performs rule-based filtering over approved student records. The Verification Agent checks the evidence required by the institutional rules before a student is treated as verified eligible. This separation helps distinguish a preliminary match from a verified result.		

Application	and	Communication	Automation
The Application Agent prepares the company-specific application workflow. The Communication Agent sends the appropriate email or notification and			

tracks whether the student has applied. Follow-ups can be targeted to pending students instead of being sent to everyone.

Assessment and Shortlisting

The Assessment Agent receives authorized results and normalizes them for the drive. The Shortlisting Agent combines verified eligibility, application state and assessment conditions to prepare a shortlist according to the approved company criteria.

Adaptive Scheduling

The Scheduling Agent considers multiple constraints before creating an interview plan. The Conflict Checker validates the proposal. If a conflict is found, the orchestrator can send the state back to scheduling with the new constraint, rather than forcing the placement team to rebuild the plan manually.

Human-in-the-Loop Control

Placify is designed to automate coordination while preserving human authority. A TPO can review exceptions, approve a final shortlist and release a schedule. This boundary makes the academic prototype more realistic and suitable for institutional use.

4.3 Agent-Based Approach

The project divides the overall placement problem into specialized responsibilities. This allows deterministic operations such as filtering and database updates to remain separate from language-based interpretation and planning. The orchestrator connects the specialists through shared state and defined transitions, making the workflow easier to test, monitor and extend.

5. Dataset / Knowledge Sources

5.1 Student and Academic Data

Placify requires structured student information such as student identifier, name, department, graduation batch, CGPA, school marks, arrear status and other institution-approved attributes. The academic/result source is treated as an authorized service. For a prototype, a controlled dataset can simulate the same fields and service responses without connecting to a real college portal.

5.2 Company and Placement Data

Company records contain the role, job description, eligibility conditions, application deadline, assessment requirements, interview requirements and recruiter contact details. Each placement drive maintains its own state so that multiple companies can be processed without mixing candidates or schedules.

5.3 Schedule Data

Scheduling requires examination dates, classes or institutional events, other placement drives, interview slots, rooms, panels and relevant student availability. The scheduling model treats these as constraints rather than as independent records.

5.4 Knowledge Sources

The Agentic RAG knowledge base can contain placement policies, eligibility guidelines, company JDs, application instructions, assessment rules, interview procedures and approved institutional documents. Documents are chunked, embedded and indexed so that agents can retrieve only the context required for the current decision.

5.5 Data Preparation

Data preparation includes schema validation, normalization of department and degree names, standardization of CGPA and mark formats, duplicate detection and handling of missing values. Each transformation should preserve traceability so that a placement officer can understand how an eligibility result was obtained.

6. Tools and Technologies Used

6.1 Technology Stack

Layer	Technology	Purpose
Programming	Python	Core implementation and agent/tool services
Orchestration	LangGraph + LangChain	Stateful workflow, agents, retrieval and tool integration
LLM	LLM service	Reasoning, planning, interpretation and explanation
Backend	FastAPI	APIs and service endpoints
Database	PostgreSQL	Structured student, company, application and schedule data
Knowledge Retrieval	Embeddings + Vector Database	Semantic retrieval for Agentic RAG
Frontend	React	Role-based web interface
Integration	MCP + APIs	Controlled access to external tools and services
Communication	Email / Notification API	Invitations, reminders and schedule updates
Forms	Form Service / Google Forms API	Company-specific application collection

6.2 Technology Selection

Python provides the main implementation environment. LangGraph is used to model the stateful agent workflow, while LangChain supports model, retrieval and tool integration. FastAPI provides backend APIs. PostgreSQL can store structured student, company, application and schedule records. A vector database supports semantic retrieval for Agentic RAG. React can provide a role-based user interface, while controlled API connectors provide forms, email, notifications, academic verification and calendar functions.

6.3 Agentic RAG Technology

The retrieval layer converts placement documents into embeddings and stores them in a vector index. At runtime an agent constructs a query from the current task and retrieves relevant policy or JD information. The retrieved evidence is passed to the LLM together with structured state so that the response is grounded in institutional knowledge.

6.4 Supporting Tools

MCP-compatible tools can expose external services through a consistent tool interface. Email and notification services handle communication, form services manage application collection, and calendar or scheduling services provide availability information. The project should keep credentials and permissions outside prompts and enforce role-based access at the service layer.

7. Agent Architecture

7.1 Architecture Overview

The Placify architecture is built around a central LangGraph Orchestrator. Data sources provide student, academic, company and schedule information. The orchestrator routes tasks to the agent layer, where each agent performs a specific placement function. The knowledge and memory layer provides contextual information and maintains state. The tool and integration layer allows agents to interact with external services. Users interact through interfaces appropriate to their role.

7.2 Architecture Diagram

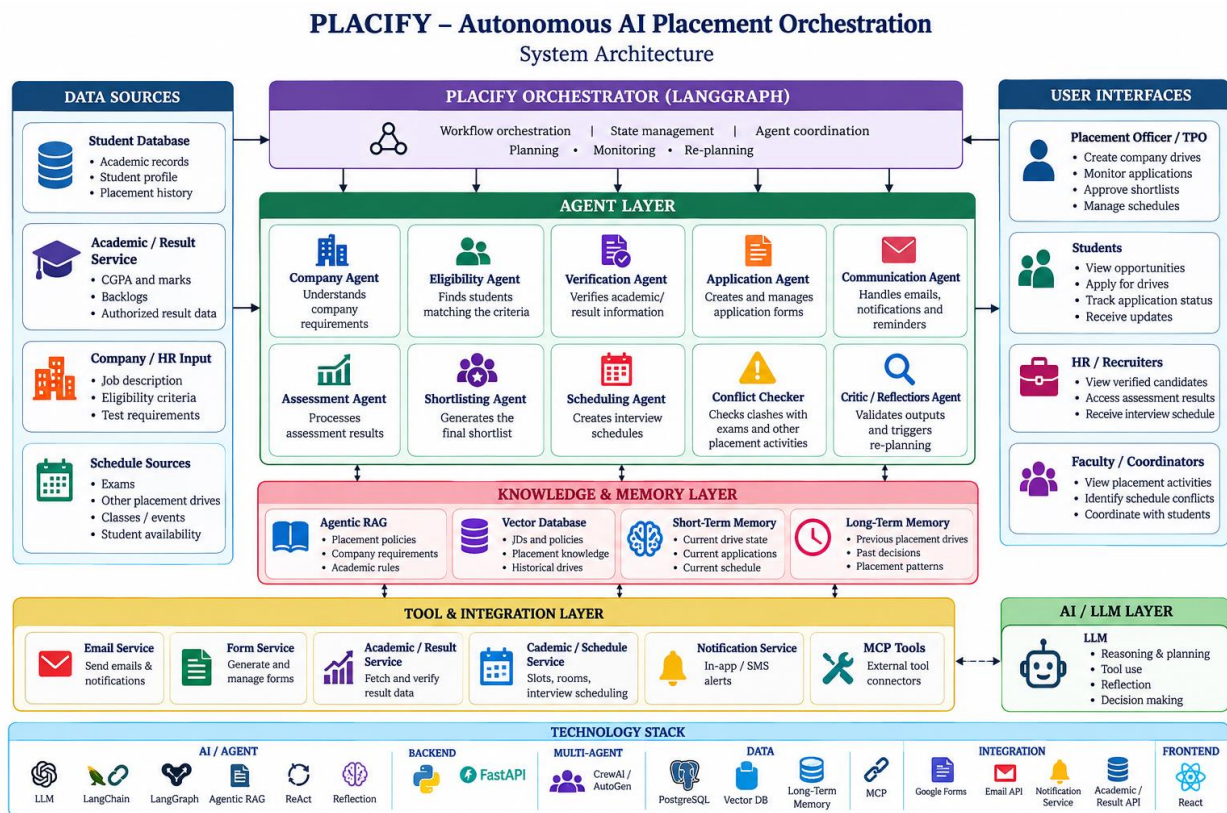


Figure 7.2.1: PLACIFY system architecture showing data sources, LangGraph orchestration, specialist agents, knowledge and memory layers, tools/integrations, AI/LLM layer, user interfaces, and technology stack.

7.3 Architecture Principles

The architecture follows five principles: stateful coordination, specialized agents, tool-based action, grounded retrieval and human approval. The LLM is used for interpretation, reasoning and planning, while deterministic services perform operations that require predictable results such as eligibility filtering, database updates and schedule validation.

8. Agent Architecture – Agent Responsibilities

8.1 Agent Components

The Placify agent architecture consists of a central orchestrator and ten specialist agents. Each specialist receives only the state required for its responsibility and returns a structured result. The orchestrator records the result and determines the next transition.

8.2 Agent Responsibilities

Company Agent

Extracts role, department, academic criteria, skills, assessment requirements and important dates from company input. It converts unstructured JD information into structured requirements.

Eligibility Agent

Filters the student population against the structured criteria and identifies the candidate set. It reports the conditions used for the match.

Verification Agent

Checks important eligibility attributes against an authorized academic/result source and distinguishes verified students from records requiring review.

Application Agent

Creates or configures the company-specific application workflow and associates responses with the correct placement drive.

Communication Agent

Sends invitation emails, reminders, deadline messages and schedule updates. It uses application state so that reminders can target only students who still need to act.

Assessment Agent

Processes authorized test or assessment results and associates scores or pass/fail status with the correct candidate and drive.

Shortlisting Agent

Applies the approved shortlist criteria to verified candidates and prepares a structured candidate list for review.

Scheduling Agent

Constructs interview schedules from candidate availability, exams, other placement events, rooms and panel constraints.

Conflict Checker

Validates proposed schedules and identifies clashes or invalid assignments. Its output can trigger another scheduling attempt.

Critic / Reflection Agent

Reviews intermediate outputs for missing information, contradictory state or scheduling violations. It can return the workflow to the relevant agent for correction.

8.3 Shared State

Shared state can include company identity, role, eligibility criteria, eligible student IDs, verification status, application status, assessment results, shortlist state, schedule state, conflict reports, approval status and audit events. Keeping these elements in one state model allows agents to cooperate without repeatedly reconstructing the entire placement context.

9. Agent Workflow and Orchestration

9.1 Workflow Design

The placement workflow begins when a company drive is created. The Company Agent structures the requirements and the Eligibility Agent identifies candidates. Verification then confirms the relevant academic evidence. Once verified, the Application Agent opens the application process and the Communication Agent manages invitations and reminders. After assessment, the Shortlisting Agent prepares the candidate list. Scheduling and conflict checking then operate on the final candidate set.

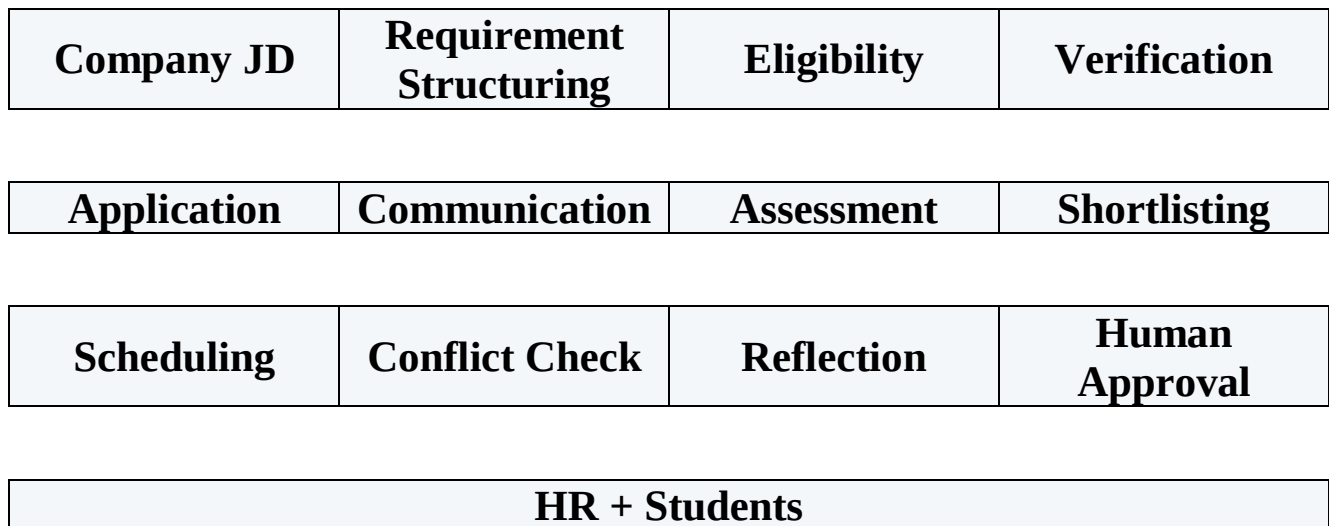


Figure 9.1.1: End-to-end Placify workflow from company requirement to approved interview schedule.

9.2 Task Decomposition

Task decomposition ensures that each stage has a clear responsibility. Requirement interpretation is separated from eligibility filtering; verification is separated from communication; and scheduling is separated from conflict validation. This separation makes it possible to replace one component without redesigning the entire workflow.

9.3 Agent Coordination

Agent coordination is implemented as a state transition process. The orchestrator maintains the current drive state and routes outputs to the next appropriate node. If the Eligibility Agent reports missing criteria, the workflow can request clarification rather than inventing a condition. If the

Conflict Checker finds a schedule violation, the state returns to the Scheduling Agent with the detected constraints.

9.4 Orchestration Implementation

LangGraph represents the workflow as connected nodes and conditional transitions. LangChain can provide reusable interfaces for prompts, retrieval and tools. Deterministic functions should handle operations such as database queries, eligibility rules and conflict calculations, while the LLM handles interpretation, planning and explanation.

9.5 Decision Routing

Condition	Routing Behaviour
Company criteria received	Invoke Company Agent to structure requirements.
Criteria incomplete	Request clarification or mark the drive for TPO review.
Eligibility conditions ready	Invoke Eligibility Agent.
Academic evidence required	Invoke Verification Agent through authorized service.
Applications open	Application + Communication Agents begin the application cycle.
Assessment complete	Invoke Assessment and Shortlisting Agents.
Shortlist ready	Send to Scheduling Agent.
Schedule conflict detected	Return state to Scheduling Agent with conflict constraints.
Plan validated	Request human approval before release.

9.6 Error Handling and Recovery

The system should distinguish incomplete data from a negative eligibility result. If an authorized result service is unavailable, the candidate should be marked as pending verification rather than incorrectly rejected. If retrieval fails, the agent should report that grounding was unavailable. If scheduling fails, the system should retain the current state and request a new plan instead of silently publishing an invalid schedule.

10. Agentic RAG Implementation

10.1 Retrieval Architecture

Agentic RAG provides the knowledge-grounding layer of Placify. The knowledge corpus can contain placement policies, eligibility rules, company JDs, application instructions, assessment guidance and interview procedures. Documents are cleaned, split into meaningful chunks, enriched with metadata and converted into embeddings before being stored in a vector index.

Placement Policies	Company JDs	Chunk + Metadata	Embeddings + Vector DB	Agentic Retrieval	LLM + Decision
---------------------------	--------------------	-------------------------	-------------------------------	--------------------------	-----------------------

Figure 10.1.1: Agentic RAG pipeline for placement policies, company JDs and institutional guidance.

10.2 Retrieval Pipeline

- Collect and curate approved placement documents.
- Split documents into policy, eligibility, application and scheduling sections.
- Attach metadata such as document type, company, role and revision.
- Generate embeddings and store them in the vector index.
- Construct runtime queries from the current agent goal and placement state.
- Retrieve relevant chunks and pass them to the reasoning stage.

10.3 Knowledge Retrieval and Grounding

At runtime, the retrieval query can combine the company, role, student condition and current task. For example, when the Eligibility Agent needs to interpret an institution rule concerning arrears, it can retrieve the relevant policy instead of relying on a hardcoded assumption. The retrieved source is preserved in the decision context.

10.4 Generation Process

The LLM receives structured placement state together with retrieved evidence. The system should distinguish verified facts, retrieved policy text and generated reasoning. This makes it possible to present a recommendation as a decision with supporting evidence rather than as an unexplained model response.

10.5 Retrieval-Quality Evaluation

Retrieval can be evaluated through top-k relevance, source coverage and the rate of unsupported responses. Test queries should cover eligibility, application policy, assessment rules and scheduling questions. Results should be recorded during the final experimental run rather than being invented in advance.

10.6 RAG Safety Considerations

Placement documents may change across recruitment cycles. The knowledge base should therefore retain source and revision information. Sensitive student records should not be placed in unrestricted retrieval collections. Access controls should be applied to both the documents and the tools that can act on retrieved information.

11. System Implementation

11.1 Functional Prototype

The Placify prototype can be implemented as a modular web application in which the user interface communicates with backend services and the agent orchestrator. A placement officer creates a drive, uploads or enters company requirements and reviews the system's generated eligibility conditions. The dashboard then exposes the current state of applications, assessments, shortlisting and scheduling.

ACTIVE DRIVES 4	ELIGIBL E 842	APPLICA TIONS 617	SHORTL ISTED 214	CONFLI CTS 7	PENDIN G APPROV AL 3
--------------------------------	------------------------------	----------------------------------	---------------------------------	-----------------------------	---

Company	Stage	Applications	Conflicts	Status
ABC Technologies	Shortlisting	287	2	Review
Nova Systems	Interview	156	0	Approved
Orbit Labs	Applications	174	5	Active

Figure 11.1.1: Placement control-center interface showing drive state, applications, conflicts and pending approvals.

11.2 Drive State Management

Each company drive maintains a unique state containing company information, eligibility criteria, candidate sets, application status, assessment status, shortlist status and schedule status. State transitions are logged so that the TPO can understand what the system has done and what remains pending.

11.3 Eligibility and Verification

The eligibility pipeline first normalizes the company requirements. Structured rules then filter the student dataset. The Verification Agent requests the required academic evidence through an authorized interface. Records that cannot be verified are held in a review state rather than being silently classified as eligible or ineligible.

Company Criteria	Student DB	Eligibility Rules	Authorized Result	Verified State
-----------------------------	-------------------	------------------------------	------------------------------	---------------------------

Figure 11.3.1: Eligibility verification flow.

11.4 Application and Communication

After verification, the Application Agent creates the company-specific application workflow. The Communication Agent sends invitations and records delivery and response status. A pending-state query identifies students who have not responded, allowing targeted reminders before the deadline.

11.5 Assessment and Shortlisting

Assessment results are associated with the relevant drive and candidate identifiers. The Shortlisting Agent applies the approved rules and produces a structured shortlist. The TPO can inspect the evidence and approve the final list before it is shared with HR.

11.6 Interview Scheduling

Scheduling is treated as a constraint problem. The Scheduling Agent receives the shortlisted candidates and relevant calendar information. It attempts to allocate interview slots while respecting exam dates, other placement drives, room capacity, panel availability and student availability. The Conflict Checker validates the result.

Shortlist	Examinations	Other Drives	Availability
------------------	---------------------	---------------------	---------------------

Scheduling Agent	Conflict Checker	Approved Schedule
-------------------------	-------------------------	--------------------------

Figure 11.6.1: Conflict-aware interview scheduling flow.

11.7 Adaptive Re-planning Scenario

Suppose HR moves an interview from 10 AM to 2 PM. The orchestrator records the change, identifies the affected candidates and sends the updated constraints to the Scheduling Agent. The new schedule is checked again. If conflicts remain, the Critic/Reflection Agent sends the workflow back for another planning cycle. Only after validation and approval is the updated schedule communicated.

11.8 Human Approval and Audit

Placify records important transitions such as eligibility verification, shortlist approval and schedule release. Human approval is retained for decisions that can affect students or recruiter commitments. This creates an audit trail and prevents an LLM response from being treated as an automatically authorized institutional action.

11.9 Project Plan and Implementation Milestones

Phase	Milestone / Deliverable	Target
Phase 1	Requirement analysis, placement workflow mapping and controlled dataset design.	Weeks 1–2
Phase 2	Agent architecture, state model, database and RAG design.	Weeks 3–4
Phase 3	Eligibility, verification, application and communication agents.	Weeks 5–7
Phase 4	Assessment, shortlisting and scheduling agents with conflict checking.	Weeks 8–10
Phase 5	Reflection loop, tool integration, dashboard and audit state.	Weeks 11–13
Phase 6	Testing, evaluation, documentation and final demonstration.	Weeks 14–16

11.10 Prototype Boundary

The prototype demonstrates the orchestration and decision-support workflow. It should not be represented as having unrestricted access to a real college result website or institutional database unless such access has been formally authorized. Controlled mock services can be used to demonstrate the same API interactions safely.

12. Experimental Evaluation and Results

12.1 Evaluation Metrics

Evaluation of Placify should cover both individual components and the complete agentic workflow. A placement system may produce correct eligibility results while still failing if communication is not triggered, state is lost or scheduling creates conflicts. Therefore the evaluation includes data correctness, retrieval quality, workflow reliability, communication tracking and scheduling quality.

KPI	What It Measures
Eligibility Accuracy	Correctness of eligible/ineligible classification against the authorized test data.
Verification Consistency	Agreement between the stored student record and the authorized result response.
Application Tracking Accuracy	Correct identification of applied, pending and completed responses.
Shortlist Consistency	Whether candidates satisfy the approved shortlist conditions.
Schedule Conflict Rate	Number of published schedules containing detected conflicts.
RAG Relevance	Whether retrieved policies and JDs are relevant to the current agent task.
Workflow Completion	Percentage of controlled placement scenarios completed successfully.

12.2 Testing Methodology

Testing should use controlled placement scenarios with different eligibility rules, student records and schedule constraints. Each scenario records the input state, agent actions, tool calls, retrieved context, decisions, conflicts and final state. Failure cases should include missing academic data, incomplete company criteria, duplicate applications, changed interview timing and unavailable calendar information.

12.3 Performance Results

The present report does not claim verified numerical performance measurements for Placify because the project material supplied for this report does not contain final experimental benchmark values. Therefore numerical accuracy, precision, recall or latency values should not be invented. The final implementation can populate the evaluation table after controlled experiments are run.

Evaluation Area	Evidence	Final Value
Eligibility	Controlled ground-truth comparison	To be populated from final run
Verification	Success/failure cases	To be populated from final run
Communication	Invitation/reminder transitions	To be populated from final run
Shortlisting	Rule-compliant candidate list	To be populated from final run
Scheduling	Conflict-free schedule rate	To be populated from final run
RAG	Top-k relevance/source coverage	To be populated from final run
Agent Workflow	End-to-end scenario completion	To be populated from final run

12.4 Qualitative Functional Validation

A successful demonstration should show a company requirement entering the system, eligible students being identified and verified, applications being initiated, responses being tracked, a shortlist being generated and an interview schedule being created without conflicts. A second scenario should change the interview timing and show the re-planning loop.

12.5 Result Interpretation

The system should be assessed as an end-to-end orchestration system rather than only as a chatbot. Reliability of state transitions, evidence grounding, tool execution, conflict handling and human approval are important parts of the final evaluation.

13. Comparison, Innovation and Scalability

13.1 Comparison with Baseline

A conventional placement workflow generally separates eligibility checking, forms, communication, assessment processing, shortlisting and scheduling. Staff members move information manually between these stages. Placify connects the same stages through a stateful multi-agent workflow.

Traditional / Fragmented Workflow	Placify Workflow
Manual eligibility checking	Eligibility Agent applies structured criteria
Separate academic verification	Verification Agent checks authorized data
Manual forms and invitations	Application + Communication Agents coordinate the cycle
Manual reminder lists	Communication Agent targets pending applicants
Manual shortlist preparation	Assessment + Shortlisting Agents combine conditions
Manual schedule reconciliation	Scheduling Agent + Conflict Checker validate constraints
Restarting after changes	Orchestrator preserves state and re-plans

13.2 Advantages of the Proposed System

- Connects the full placement lifecycle instead of automating only one form.
- Separates specialist responsibilities so that the workflow remains modular.
- Uses Agentic RAG to ground decisions in placement knowledge.
- Maintains short-term and long-term placement context.
- Supports targeted communication based on application state.
- Detects scheduling conflicts before the schedule is released.
- Retains human approval and auditability for important decisions.

13.3 Innovation

The main innovation of Placify is the combination of placement eligibility, verification, communication, assessment, shortlisting and adaptive scheduling inside one agentic workflow. Conventional automation often performs one fixed operation at a time. Placify instead maintains the current state of a drive and selects the next action based on that state.

A second innovation is the re-planning capability. A schedule is treated as a plan that can be checked and revised. When HR changes an interview time, the system does not discard the complete drive state. It identifies the affected portion, regenerates the schedule under the new constraints and validates the result.

A third innovation is the combination of Agentic RAG and institutional memory. Agents can retrieve the placement policy relevant to a question while preserving the context of the current drive. This makes the workflow more explainable than a standalone generative assistant.

13.4 Scalability

The architecture can scale by adding more students, companies, drives and institutional documents while keeping the same agent roles. A larger deployment can use a production relational database, a managed vector store, enterprise identity management and institution-approved APIs. Additional agents could support interview analytics, placement statistics, recruiter feedback or student preference matching.

13.5 Extensibility Roadmap

Extension	Purpose
College ERP Integration	Synchronize approved student and academic records
Recruiter Platform Integration	Receive authorized assessment and interview updates
Advanced Scheduling	Optimize room, panel and student constraints
Placement Analytics	Generate institution-level insights
Preference-Aware Matching	Use approved student role preferences

13.6 Real-World Applicability

Placify can be applied conceptually to engineering colleges, universities and other institutions that conduct structured recruitment drives. The architecture is most useful where eligibility data, placement policies and schedules are available through controlled digital systems. The same pattern can be extended to internships, project recruitment, industry training and other student selection workflows.

13.7 Security and Responsible AI

Student information is sensitive and should be accessed only by authorized roles. Credentials and API keys should be stored securely and should never be embedded in prompts. Role-based access should control what students, TPOs, recruiters and faculty can see. Audit logs should record important actions and state transitions.

Responsible AI also requires transparency about uncertainty. If an academic record cannot be verified, the system should say that verification is pending rather than guessing. If an agent proposes a schedule, the system should expose the constraints used for validation. Critical decisions should remain subject to institutional approval.

13.8 Limitations

The academic prototype depends on the quality and availability of institutional data. Real-world deployment would require authorized integrations, privacy controls, authentication, service reliability and extensive testing. The system should not be treated as a replacement for official placement policies or human decision authority.

14. Conclusion and Future Work

14.1 Conclusion

Placify presents an Agentic AI approach to campus placement coordination in which company requirements, student eligibility, verification, communication, assessment, shortlisting and interview scheduling are connected through a stateful multi-agent workflow. The project moves beyond the idea of a static placement portal by giving specialized agents clear responsibilities and allowing an orchestrator to decide which step should happen next.

The Company Agent structures recruiter requirements, the Eligibility and Verification Agents identify and validate candidates, the Application and Communication Agents coordinate student interaction, and the Assessment and Shortlisting Agents prepare the candidate pipeline. The Scheduling Agent and Conflict Checker handle the constraints around interviews, while the Critic/Reflection Agent validates the plan and supports re-planning. Agentic RAG and memory provide the contextual information required by these agents.

A key contribution is the adaptive workflow. When an external event changes, such as an interview timing update from HR, the system can preserve the existing drive state, identify affected tasks, generate an updated plan and validate it before communication. This demonstrates a practical use of Agentic AI beyond a question-answering chatbot.

The architecture is designed as an academic prototype and deliberately retains human approval for critical actions. Real institutional deployment would require authorized data integrations, security controls, privacy governance and validation with the placement cell. Within these boundaries, Placify demonstrates how reasoning, tool use, memory, retrieval, reflection and multi-agent collaboration can be combined to automate a realistic college workflow.

14.2 Future Work

- Integrate with approved college ERP and academic-result services.
- Add stronger identity and role-based access controls.

- Support multiple simultaneous company drives with shared resource constraints.
- Introduce advanced constraint optimization for large-scale interview scheduling.
- Add recruiter feedback and placement analytics.
- Expand the knowledge base with versioned institutional policies.
- Add student preference-aware opportunity discovery where institutionally appropriate.
- Evaluate the system using larger controlled datasets and real placement scenarios.
- Add monitoring for agent failures, tool failures and inconsistent state transitions.

14.3 Final Perspective

Placify is intended to show that Agentic AI can be applied to a real institutional workflow where the important challenge is not simply generating text, but coordinating multiple decisions, tools, constraints and stakeholders over time. The project therefore positions the placement cell as the human authority and the agentic system as an intelligent orchestration layer that reduces repetitive work while keeping important decisions traceable.